

LUIS ANGEL VALENCIA

Insertar "Error Critico" inicio

$H \rightarrow \text{Null}$ $T \rightarrow \text{Null}$

Nuevo nodo

$nd = [\text{null} | \text{error critico} | \text{null}]$

$H \rightarrow [\text{null} | \text{error critico} | \text{null}] \leftarrow T$

insertar "Aviso Pago"

$nd = [\text{null} | \text{Aviso Pago} | S]$

↓

$[nd | \text{error critico} | \text{null}]$

$H \rightarrow [\text{null} | \text{Aviso Pago} | S] \leftarrow [S | \text{error critico} | \text{null}] \leftarrow T$

Insertar "Resumen" inicio

$H \rightarrow [\text{null} | \text{Aviso Pago} | S] \leftarrow [S | \text{error critico} | \text{null}] \leftarrow T$

$nd = [\text{null} | \text{Resumen} | S]$

Imprimir reverso

$T \rightarrow [S | \text{error critico} | \text{null}]$

Eliminar al final

$H \rightarrow \text{Resumen} \leftarrow \text{Aviso Pago} \leftarrow \text{error critico} \leftarrow T$

$T \rightarrow [S | \text{Aviso Pago} | S]$

LUIS ANGEL VALENCIA

```
public static void main (String[] args) {  
    List<Double> messages = new ArrayList<>();  
    messages.add(1.0);  
    messages.add(2.0);  
    messages.add(3.0);  
    messages.add(4.0);  
    messages.add(5.0);  
    System.out.println("Impression normal");  
    messages.forEach(System.out::println);  
    System.out.println("Impression inverse");  
    messages.forEach(System.out::println);  
    System.out.println("Buscar mensaje");  
    messages.add(6.0);  
    System.out.println("Lista actualizada");  
    messages.forEach(System.out::println);  
}
```

LUIS ANGEL VALENCIA

```
public void eliminarFinal() {  
    if (cola == null) {  
        System.out.println("Lista vacia :(");  
        return;  
    }  
    if (cabeza == cola) {  
        cabeza = null;  
        cola = null;  
    } else {  
        cola = cola.getAnterior();  
        cola.setSiguiente(null);  
    }  
}
```

Se falta el
gratificación

Exámenes de Ptes, Colas, Ordenamiento
Enunciado Lista Ptes, Colas, Ordenamiento

```
public void imprimirLista() {  
    Nodo<T> actual = cabeza;  
    System.out.println("Lista normal : ");  
    while (actual != null) {  
        System.out.println(actual.getData() + " ");  
        actual = actual.getSiguiente();  
    }  
}
```

```
public void imprimirInverso() {  
    Nodo<T> actual = cola;  
    System.out.println("Lista inversa");  
    while (actual != null) {  
        System.out.println(actual.getData() + " ");  
        actual = actual.getAnterior();  
    }  
    System.out.println("null")  
}
```

LUIS ANGEL VALENCIA

```
public Node<T> get siguiente() {  
    return siguiente;  
}  
public void set siguiente (Node<T> siguiente) {  
    this.siguiente = siguiente;  
}  
public Node<T> get anterior() {  
    return anterior;  
}  
public void set anterior (Node<T> anterior) {  
    this.anterior = anterior;  
}  
}
```

```
public class ListaDoble<T> {  
    private Node<T> cabeza;  
    private Node<T> cola;  
}
```

```
public ListaDoble() {  
    this.cabeza = null;  
    this.cola = null;  
}
```

```
public void agregarNodo(T dato) {  
    Node<T> nuevo = new Node<T>(dato);  
    if (cabeza == null) {  
        cabeza = nuevo;  
        cola = nuevo;  
    } else {  
        nuevo.set siguiente (cabeza);  
        cabeza.set anterior (nuevo);  
        cabeza = nuevo;  
    }  
}
```

3.3 Recorrer/imprimir hacia atras

```
public void imprimirReves() {  
    Nodo <T> actual = Cola;  
    System.out.print ("Lista en Reverso : ");  
    while  
        (actual != null) {  
        System.out.print (actual.getData() + " - ");  
        actual = actual.getAnterior();  
    }  
    System.out.println ("null");  
}
```

4 Caso practico : Gestion de una Cola de Mensajes

```
Public class Nodo <T> {  
    private T dato;  
    private Nodo <T> siguiente;  
    private Nodo <T> anterior;  
  
    public Nodo (T dato) {  
        this.dato = dato;  
        this.siguiente = null;  
        this.anterior = anterior;  
    }  
  
    public T getData() {  
        return dato;  
    }  
  
    public void setDato (T dato) {  
        this.dato = dato;  
    }  
}
```

LUIS ANGEL VALENCIA

Operaciones clave

Deafio actualizar ambas referencias

3.1 Agregar al inicio

```
public void agregarInicio(T dato) {  
    Nodo<T> nuevo = new Nodo<>(dato);  
    if (cabeza == null) {  
        cabeza = nuevo;  
        cola = nuevo;  
    } else {  
        nuevo.setSiguiente(cabeza);  
        cabeza.setAnterior(nuevo);  
        cabeza = nuevo;  
    }  
}
```

3.2 Eliminar al final

```
public void eliminarFinal() {  
    if (cola == null) {  
        System.out.println("Lista vacia");  
        return;  
    }  
    if (cabeza == cola) {  
        cabeza = null;  
        cola = null;  
    } else {  
        cola = cola.getAnterior();  
        cola.setSiguiente(null);  
    }  
}
```


LUIS ANGEL VALENCIA

Luis Angel Valencia Jages
Implementacion lista doblemente enlazada

Estructura de clase
Implementar Tipos Genericos y Encapsulamiento

```
Public class Node <T> {  
    Private T dato;  
    Private Node <T> siguiente;  
    Private Node <T> anterior
```

// Constructor

```
public Node(T dato) {  
    this.dato = dato;  
    this.siguiente = null;  
    this.anterior = null;  
}
```

// setter and getter
}

```
Class ListDoble <T> {  
    public class ListDoble <T> {  
        Private Node <T> cabeza;  
        Private Node <T> cola;
```

```
    public ListDoble() {  
        this.cabeza = null;  
        this.cola = null;  
    }
```

// operations
}

LUIS ANGEL VALENCIA

4 Caso Práctico: Gestion de un historial de Transacción

```
Public class Main {  
    public static void main(String[] args) {  
        lista = new LinkedList();  
        System.out.println("Historial de Transacciones");  
        history.agregarInicio(150);  
        history.agregarInicio(45);  
        history.agregarInicio(200);  
        history.agregarInicio(75);  
  
        System.out.println("Historial de Transacciones actual");  
        history.imprimirLista();  
        System.out.println("Cancelar Transacción serializada (75)");  
        history.eliminarInicio();  
        System.out.println("Historial de cancelaciones");  
        history.imprimirLista();  
    }  
}
```

Desafío implementar operacion Agregar al final

```
Public void agregarFinal(int dato) {  
    Nodo nuevo = new Nodo(dato);  
    if (cabecera == null) {  
        cabecera = nuevo;  
        return;  
    }  
    Nodo actual = cabecera;  
    while (actual.getSiguinte() != null) {  
        actual = actual.getSiguinte();  
    }  
    actual.setSiguinte(nuevo);  
}
```


LUIS ANGEL VALENCIA

3.1 Agregar al Inicio (agregarAlInicio(int dato))

```
public void agregarAlInicio(int dato) {  
    Nodo nuevo = new Nodo(dato);  
    nuevo.setSiguiente(cabeza);  
    cabeza = nuevo;  
}
```

3.2 Eliminar al inicio

```
public void eliminarInicio() {  
    if (cabeza != null) {  
        cabeza = cabeza.getSiguiente();  
    }  
}
```

3.3 Recorrer / imprimir

```
public void imprimirLista() {  
    Nodo actual = cabeza;  
    while (actual != null) {  
        System.out.println(actual.getData() + " ");  
        actual = actual.getSiguiente();  
    }  
    System.out.println("Fin de la lista");  
}
```

LUIS ANGEL VALENCIA

3 Dibujo

Inserta 150 inicio
 $H \rightarrow \text{Null}$

Nuevo Nodo

$N_d = [150 | \text{Null}]$

$H \rightarrow [150 | \text{Null}]$

Inserta 45 inicio

$H \rightarrow [150 | \text{Null}]$

$N_d = [45 | s]$

$[45 | s] \rightarrow [150 | \text{Null}]$

$H \rightarrow [45 | s] \rightarrow [150 | \text{Null}]$

200 al inicio

$H \rightarrow [45 | s] \rightarrow [150 | \text{Null}]$

$N_d = [200 | s]$

$H \rightarrow [200 | s] \rightarrow [45 | s] \rightarrow [150 | \text{Null}]$

75 al inicio

$H \rightarrow [200 | s] \rightarrow [45 | s] \rightarrow [150 | \text{Null}]$

$N_d = [75 | s]$

LUIS ANGEL VALENCIA

```
if (actual == null) {  
    return false;  
}  
anterior.setSiguiente(actual, getSiguiente());  
return true;  
public void Ver() {  
    Nodo aux = cabeza;  
    while (aux != null) {  
        System.out.println(aux.getData() + " - ");  
        aux = aux.getSiguiente();  
    }  
}
```

Encapsulamiento con datos genericos

```
public class ListaGenerico <T> {  
    private Nodo cabeza <T>;
```

```
    public void agregarPrimario (T dato) {
```

```
    }  
    public void agregarFin (T dato) {
```

```
    }  
    public boolean eliminar (T dato) {
```

```
    }  
    Nodo <T> actual = cabeza;  
    Nodo <T> anterior = null;
```

LUIS ANGEL VALENCIA

```
// agregamos metodos
public void agregarInicio(int dato) {
    Nodo nuevo = new Nodo(dato);
    nuevo.setSiguiente(cabeza);
    cabeza = nuevo;
}

public void agregarFin(int dato) {
    Nodo nuevo = new Nodo(dato);
    if (estVacio()) {
        cabeza = nuevo;
        return;
    }
    Nodo aux = cabeza;
    while (aux.getSiguiente() != null) {
        aux = aux.getSiguiente();
    }
    aux.setSiguiente(nuevo);
}

public boolean eliminar(int dato) {
    if (estVacio()) {
        return false;
    }
    if (cabeza.getDato() == dato) {
        cabeza = cabeza.getSiguiente();
        return true;
    }
    Nodo actual = cabeza;
    Nodo antes = null;
    while (actual != null && actual.getDato() != dato) {
        antes = actual;
        actual = actual.getSiguiente();
    }
}
```

LUIS ANGEL VALENCIA

```
public T getDato() {  
    return dato;  
}  
public void setDato(T dato) {  
    this.dato = dato;  
}  
public Node(T) getSiguiende() {  
    return siguiente;  
}  
public void setSiguiende(Node(T) siguiente) {  
    this.siguiente = siguiente;  
}  
}
```

Corregir clase ListaEnlazada

```
Clase ListaEnlazada  
public class ListaEnlazada {  
    Node cabeza; // referencia al primer nodo  
    public ListaEnlazada() { // constructor  
        this.cabeza = null;  
    }  
    // Metodos de operaciones  
}
```

Codigo corregido y completado

```
public class ListaEnlazada {  
    private Node cabeza;  
    public ListaEnlazada() {  
        this.cabeza = null;  
    }  
    public boolean isEmpty() {  
        return cabeza == null;  
    }  
}
```


LUIS ANGEL VALENCIA

Luis Angel Valencia Jaates GTID141

- Implementación de lista simplemente enlazada con nodos

2 Corregir las siguientes clases en Java

Clase Nodo

original

```
public class Nodo {  
    // dato cualquier que almacenara  
    int dato;  
    // siguiente  
    Nodo siguiente;  
    // Constructor  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.siguiente = null;  
        // No apunta a nada  
    }  
}
```

Código corregido

```
public class Nodo {  
    private int dato;  
    private Nodo siguiente;  
    // Constructor  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.siguiente = null;  
    }  
    // get y set  
    public int get Dato() {  
        return dato;  
    }  
    public void set dato(int dato) {  
        this.dato = dato;  
    }  
    public Nodo get siguiente() {  
        return siguiente;  
    }  
    public void set siguiente(Nodo siguiente) {  
        this.siguiente = siguiente;  
    }  
}
```

2.1 Definir el encapsulamiento en la clase Nodo y que sea Nodo con dato generico

```
public class Nodo <T> {  
    private T dato;  
    private Nodo <T> siguiente;  
  
    public Nodo (T dato) {  
        this.dato = dato;  
        this.siguiente = null;  
    }  
}
```